Dcache Analysis and Pseudo-LRU Optimization

余嘉閔, 113511060

***Abstract*—analyzing the hit/miss rate of dcache, the average latency of each hit/miss, and how often miss/hit happen continuously. I also optimize the dcache by an LRU-like optimization.**

Keywords—dcache, LRU, Replacement policy, Aquila

I. INTRODUCTION

Cache is a bridge between memory and cpu, in multicore-cpu, there can be up to 3 levels of cache. Aquila is a single-core cpu so it only have to use 1 level cache. Cache can be divided into dcache and icache, dcache is used to store general data, icache is used to store instructions, the former one is more needed to optimize it's efficiency because unlike icache, which often fetch instructions in +4 order, dcache fetch data more ramdomly. Some common optimizations of dcache are prefetch and LRU replacement policy. We will analyze the efficiency of the dcache by calculate pi to 5000 digit ,and optimize it by a pseudo LRU replacement policy.

II. PSEUDE LRU REPLACEMENT POLICY

A. Overview

When cache miss it have to add new data to the line according to the index, therefore, a block in the line have to be the victim to be replace. The original replacement policy is rotate around all the ways, regardless of which way is more frequently used. The policy has a shortcoming that if a way in the line is used frequently is might still be replace because now is its turn.

B. The mechanism and way to realize

First, I use a Two-dimensional table, by declare

```
reg [2:0] lru[LINE_BITS-1:0][N_WAYS-1:0];
```

The table is used to record the usage rate of each block, with score 0 to 7, and the default score is 2, the score will change while cache hit and miss.

How to maintain the table:

When cache hit:

the score of the block being hit +1, and the score of the other blocks in the same line -1.

When cache miss:

we first find the way with the minimal lru score in the line corresponding to the index now, and we set it be the victim to be replace. Second, the score of other blocks in the same line all -1, except from the victim. Third, after the main memory send back the data and store it in the victim block, the default score of the block is set to be 2.

III. ANALYSIS

A. The data I record:

a. Total write times

b. Total read times

c. Total cycles when write: How many cycle is spent during all cache write

d. Total cycles when read: How many cycle is spent during all cache read

e. Total hit times

f. Total miss times

g. Total cycles when miss: How many cycle is spent during all cache miss

h. The number of cache hits for which at least two of the preceding three requests were also hits.

i. The number of cache miss for which at least two of the preceding three requests were also miss.

B. The data to be analyzed

1. Avg write latency = c./a. (total cycle/total times)
2. Avg ead latency = d./b.
3. Avg hit latency = 1
4. Avg miss latency = g./f.
5. Cache Hit rate = hit times/(hit times + miss times)
6. Hit density (the rate that hits happen together) = h./e.
7. miss density (the rate that miss happen together) = i./f.

C. Analysis

	2 way	4 way	8 way	4 way with LRU	8 way with LRU
Total time (ms)	29691	29689	29688	29123	29123
write latency (cycles/write)	9.79	9.79	9.79	9.23	7.72
read latency (cycles/read)	12.73	12.73	12.73	12.36	11.57
hit latency (cycles/hit)	1	1	1	1	1
Miss latency (cycles/miss)	52.48	52.48	52.48	52.42	52.27
hit rates	80.02%	80.02%	80.02%	80.8%	82.85%
Hit density	79.6%	79.6%	79.6%	81.7%	86.8%
miss density	≈ 0	≈ 0	≈ 0	≈0	≈0
Total write times	26,150,340	26,150,340	26,150,340	26,150,340	26,150,340
Total read times	36,272,247	36,272,247	36,272,247	36,272,247	36,272,247
Total cycles when write	256,103,283	256,017,123	256,043,965	241,304,016	201,790,977
Total cycles when read	462,019,901	462,023,874	461,926,673	448,397,813	419,624,551
Total hit times	49,929,831	49,929,870	49,929,845	50,457,887	51,730,197
Total miss times	12,492,756	12,492,717	12,492,742	11,964,700	10,692,390
Total cycles when miss	655,700,597	655,618,410	655,548,051	627,279,242	558,992,941
Hits happen together	39,755,920	39,756,014	39,755,998	41,227,129	44,933,029
miss happen together	6	6	6	2	2

For single column:

- It spent about 29-30 seconds to calculate pi to 5000 digits.
- It looks like it takes more cycles to perform a read operation than a write operation, but after analyzing the simulation, I think the two operation should spent the same cycles.

  the reason:

  for one read operation vs one write operation

  1. when hit, every operation finish after 1 cycle.
  2. When miss, the two operation both sent request to mem afther 3 cycles of cpu_strobe and after the same cycles of waiting the signal the data of memory and cache become ready at the same time.(but this is in simulation, maybe the memory spents different cycles of time in real world)
  3. According to the simulation, read operation and write operation should spent the same cycles, not read spent more time.
  4. There are 2 probable reason for read to spend more cycles.
     1. the memory spents different cycles of time in real world
     2. read operation encounter miss more frequently, so on average it spents more time. (this is true, after practical operation, in 4 way dcache without LRU replacement policy, there are 8,127,154 times of read when miss, but only 4,365,563 times of write when miss, therefore, when miss, read/write=1.86, and total read/write = 1.4, so read encounter miss operation more frequently.)
  5. In reality, write operation can be even faster because we can use write buffer.
- It takes significantly more cycles when a miss happens than a hit happens.

  Because when cache miss, it have to signal the memory to fetch data and sent data back. The process to access memory is complex:

  Although ddr3 is fast, but there are some reason why access memory is slow.

  1. Dram has only one port(serial in/out)
  2. The signal need to transform many times by Memory Arbiter and MIG Memory Controller.
- Read time is larger than write time, this can be one proof that programs often has more reads than write.
- 4 KB dcache can hit 80% of cpu request.
- After calculating the probability, I found the miss is more evenly distributed than randomly distributed in hits.

  (Hit_density:here79.6% vs ramdomly distributed81%)

  With LRU, miss become more evenly distributed than without LRU.

  (Hit_density:here81.7% vs ramdomly distributed85%)

Compare different columns:

- For the original replacement policy(rotate around all ways): the data are all similar in 2, 4, 8 ways.
- If we change the replacement policy to be LRU:
  1. The total computing time becomes less.

     but 4, 8 way still have not difference
  2. Avg write/read latency becomes less, and with the number of ways *2, the latency deduce drastically.
  3. hit rate grows, and with the number of ways *2, it increases dramatically.

     (80.02% -> 80.8% -> 82.85%)
- The probable reason why LRU makes number of way Meaningful.

  LRU choose the least used block in the line to be the victim, therefore, as the number of way increases, the number of blocks in the line increase, it can compare more blocks, and choose a even less used block.

  As for why the original replacement policy, the choose of victim is regular(rotate around), not considering how frequent the block is used, so the result is increasing the way number doesn't help.

IV. MENTION IN PASSING: SOME PROBABLE REASONS WHY MY NEXYS A7 100T FPGA IS BROKEN

A. Introduction to FT2232 ic

1. It's pin7、pin8 are usb inputs that the fpga board connect it to the micro usb RXD、TXD straightforwardly without any other element in the middle .
   This may lead to the broken of FT2232 when electrostatic infusion.(because these ic are made of MOSFET, which has oxide that can be easily damaged by static electricity)

2. It has 2 channel(shared the same usb input): one can transform usb signal to be jtag(to program), the other transform usb signal to be uart.
   To send Jtag signal, we can use other port on the board to replace micro usb.

3. The FT2232 appears to be the first, most complex, and relatively large IC directly connected to the Micro-USB signals.

4. Maybe using Infrared thermal imager can find which ic on the fpga is broken.(but I'm not sure whether it's useful.)

REFERENCE

[1] https://digilent.com/reference/programmable-logic/nexys-4-ddr/reference-manual

[2] https://sites.google.com/nycu.edu.tw/nycueelabs/digital?authuser=0

[3] FT2232 DATASHEET